

MACHINE LEARNING · WEEK 1 · LECTURE NOTES

Lecture 1: Introduction to Machine Learning

What machine learning is, how machines learn from data, the main types of ML, real applications, the ML workflow and the tools of the course

Item	Details
Programme	Fall Semester 2026–2027
Module	Module 1: Foundations — Python for Data Work and Introduction to ML
Lecturer	Behnam Kiani Kalejahi · Kiani-bme@hotmail.com · office hours by appointment
Companion materials	Lecture slides (69 slides with speaker notes, animations and three embedded videos) and the Lab 1 handout
Core reading	Géron (2022) Ch. 1 · James et al. (2023) ISLP Ch. 1–2 · Müller & Guido (2016) Ch. 1

How to use these notes

These notes follow the lecture slide by slide. Each section corresponds to one slide and contains the **key points** shown on the slide followed by a fuller **explanation** — the material the lecturer covers orally. Read them after the lecture, and again before the Week 2 lab. At the end you will find a glossary, a formula sheet, self-check questions with answers and a reading list.

The one-sentence summary

Machine learning is learning from examples to make good predictions on new data. Everything in this lecture — and in this course — elaborates on that sentence.

Introduction

Welcome everyone to the first lecture of Machine Learning. Before we start, look at the image on the right: it is a handwritten digit “3”, stored as an 8-by-8 grid of numbers between 0 and 16. For a computer, this is all an image is — a table of numbers.

Today's central question is simple but deep: how could we write a program that looks at those 64 numbers and says “this is a three”? By the end of this lecture you will understand why writing that program by hand is extremely hard, and why machine learning — letting the computer learn the rule from examples — is such a powerful idea.

This lecture is the foundation for the whole course. We will define machine learning, see how learning actually works, meet the three main types of ML, look at real applications, walk through the ML workflow, and finish with the tools we will use in the lab.

Welcome to Machine Learning

Slide 2

Key points

- Lecturer: Behnam Kiani (Kiani-bme@hotmail.com), office hours by appointment
- Format: offline lectures with live demos + computer lab (Python/Jupyter/Colab)
- Prerequisites: Computer Programming; Basic Probability and Statistics

Explanation

Let me introduce the course formally. My name is Behnam Kiani and I will be your lecturer this semester. The best way to reach me is by e-mail at Kiani-bme@hotmail.com; office hours are by appointment, so please write to me first and we will agree a time. For accessibility or counselling support you can also use b.kiani@centralasian.uz or the course MS Teams channel.

The course is delivered offline, in person. Each week has a traditional lecture combined with live demonstrations, and a computer-laboratory session where you will practise with Python, Jupyter notebooks and Google Colab.

The prerequisites are Computer Programming and Basic Probability and Statistics. You do not need to be an expert — if you can write a loop and a function in Python and you know what a mean and a standard deviation are, you are ready. We will revise the Python you need in weeks 1 and 2.

The course at a glance

Slide 3

Key points

- 10 credits; 30 h lectures; 60 h laboratory classes
- 15 teaching weeks + final exam week
- 4 independent-work tasks (IWS) and 7 guided consultations (IWST)

Explanation

Here are the key numbers for the course. It is a 10-credit core module in the BSc in Artificial Intelligence. There are 30 hours of lectures and 60 hours of laboratory classes — twice as much lab time as lecture time. That tells you something important: this is a hands-on course. You learn machine learning by doing it, not only by listening.

The semester runs for 15 weeks plus the final exam week. Along the way you will complete four independent-work tasks (IWS): preparing a dataset, the midterm mini-project, a reproducible pipeline notebook, and the final capstone submission. You also have seven consultation sessions (IWST) with me, which are designed to support your midterm and capstone projects. Please use them — students who attend consultations consistently produce better projects.

Course learning outcomes: by the end you will be able to...

Slide 4

Key points

- LO1: Use Python notebooks to prepare data, train simple ML models and produce predictions

- LO2: Choose a suitable basic model and evaluate it with appropriate metrics
- LO3: Explain an ML workflow, results and limitations in a report and presentation
- LO4: Work effectively in a pair or small group to finish an ML project on time

Explanation

These are the four official learning outcomes of the course, taken from the syllabus. Every lecture, lab and assessment maps onto one or more of them.

LO1 is practical: using Python notebooks to load, clean and split data, train simple models with scikit-learn and produce predictions. LO2 is about judgement: choosing a suitable basic model — linear or logistic regression, k-nearest neighbours, decision trees, Naive Bayes or k-means — and evaluating it with the right metrics, such as MAE, RMSE, accuracy, precision, recall, F1 and the confusion matrix. LO3 is communication: explaining your workflow, results and limitations in a short report and presentation. LO4 is teamwork: completing a project in a pair or small group, on time.

Notice that only the first two outcomes are about algorithms. Half of what you will be assessed on is how clearly you communicate and how well you work with others — skills employers value just as much.

How you will be assessed

Slide 5

Key points

- Weekly lab tasks & coding exercises: 10%
- Notebook checks & project-progress submissions: 10%
- Midterm mini-project (Week 7): 30%
- Final capstone project report & code: 30%
- Final presentation & viva: 20%

Explanation

This chart shows how your final 100 points are distributed. Weekly laboratory tasks and coding exercises are worth 10 percent, and notebook checks and project-progress submissions another 10 percent. That is 20 percent for simply doing the weekly work well and on time.

The midterm mini-project in Week 7 is worth 30 percent: a notebook, a short report and a 5 to 7 minute demonstration. The final capstone project is worth 30 percent for the report and code, plus 20 percent for the final presentation and viva questions.

So 80 percent of your grade comes from projects. There is no traditional written exam: the final control is your capstone viva. This means consistent effort across the semester matters far more than cramming. Deadlines are strict — late submissions lose points according to the academic policy — and academic honesty rules apply to all code and reports.

Your 15-week roadmap

Slide 6

Key points

- Module 1 (Weeks 1–5): Foundations — Python for data work and introduction to ML
- Module 2 (Weeks 6–11): Core predictive models and clustering; midterm in Week 7

- Module 3 (Weeks 12–15): Pipelines and the capstone project

Explanation

This is the full journey of the semester, organised into three modules. Module 1, weeks 1 to 5, builds foundations: what ML is, Python and pandas for data work, preparing data, simple linear regression and classification with logistic regression, and the evaluation metrics that go with them.

Module 2, weeks 6 to 11, covers the core predictive models: k-nearest neighbours, decision trees and Naive Bayes, then comparing models and finally clustering with k-means. Week 7 is the midterm mini-project workshop and demonstration.

Module 3, weeks 12 to 15, is about pipelines and the capstone project: building reproducible pipelines, planning your final project, lab support and feedback, and then submission and presentation. Each week builds directly on the previous one, so please do not fall behind — especially in the first five weeks.

Today's learning objectives

Slide 7

Key points

- Define ML and contrast it with traditional programming
- Use core vocabulary: features, labels, model, training, prediction
- Distinguish supervised, unsupervised and reinforcement learning
- Recognise applications and when ML is (not) appropriate
- Outline the ML workflow
- Name the Python tools used in this course

Explanation

By the end of today's lecture you should be able to do six things. First, define machine learning in your own words and explain how it differs from traditional programming. Second, describe the basic vocabulary: features, labels, models, training and prediction. Third, distinguish supervised, unsupervised and reinforcement learning, and inside supervised learning, regression from classification.

Fourth, recognise real-world applications and judge whether a problem is a good fit for ML. Fifth, outline the steps of a typical ML workflow. And sixth, name the Python tools we will use this semester.

I will come back to these six objectives at the end of the lecture with a short exit ticket, so keep them in mind as we go.

Agenda

Slide 8

Key points

- 01 What is machine learning?
- 02 How machines learn
- 03 Types of machine learning
- 04 ML in the real world
- 05 The ML workflow, limits and ethics

- 06 Tools, first program and what's next

Explanation

Here is our plan for today, in six parts. Part one asks the most basic question: what is machine learning? Part two opens the black box and shows how machines actually learn from data, including two short animations. Part three introduces the main types of machine learning. Part four looks at applications in the real world, including healthcare and recommender systems. Part five walks through the ML workflow step by step, including limitations and ethics. And part six introduces the tools we will use, your first ML program, and what to expect in today's lab and in your projects.

We will have a short quiz in the middle and an exit ticket at the end. Please ask questions at any time — there are no stupid questions in the first week.

Part 1 — What is machine learning?

From hand-written rules to rules learned from data

We start with the most basic question. What exactly is machine learning, how is it different from ordinary programming, and how does it relate to artificial intelligence and deep learning? We will also see why machine learning has exploded in the last fifteen years, and take a quick tour of its history.

Warm-up: write a program that recognises a “3”

Slide 10

Key points

- Try to write rules for recognising a handwritten 3
- Real handwriting varies in slant, thickness, size and style
- Every hand-written rule has exceptions

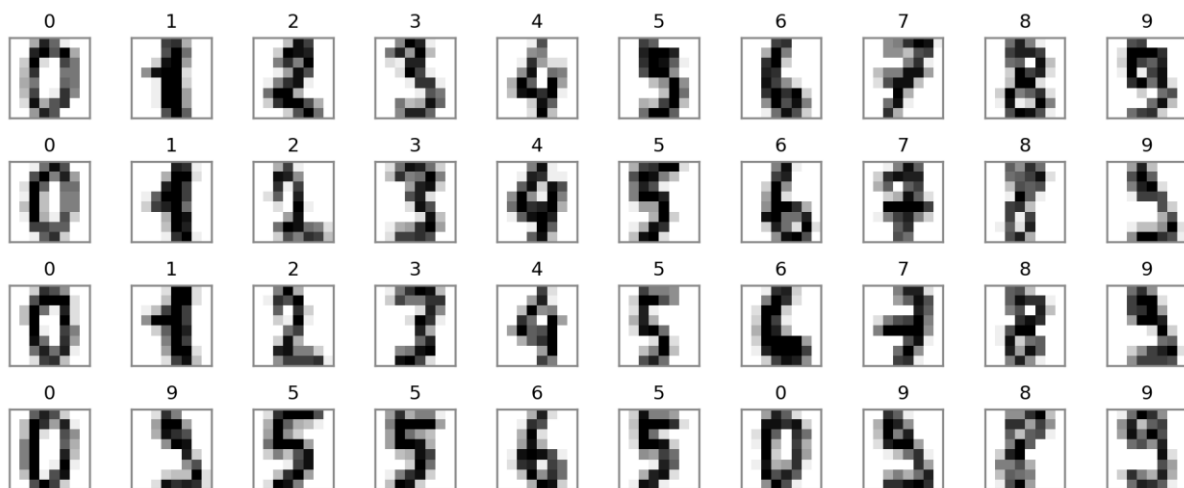


Figure: real 8×8 handwritten digits from the scikit-learn digits dataset.

Explanation

Let's start with a thought experiment. Imagine your manager asks you to write a Python function, `is_three(image)`, that returns True when an image shows a handwritten 3. These are real samples from the scikit-learn digits dataset, which we will use later in the course.

Take one minute and discuss with your neighbour: what rules would you write? Maybe “a 3 has two bumps on the right side and is open on the left”. Now look at the images: some 3s are slanted, some are thick, some have a flat top, some are written quickly and look almost like a 5 or an 8.

Most students quickly discover that every rule has exceptions, and every fix creates new exceptions. This is the key insight that motivates machine learning: for many problems, we can easily give examples of the right answer, but we cannot easily write down the rule.

Why hand-written rules fail

Slide 11

Key points

- An image is just a grid of numbers (here $8 \times 8 = 64$ values)

- Rules over many inputs become impossibly complex
- Patterns may also change over time (e.g. spam tactics)
- ML finds the rules automatically from examples

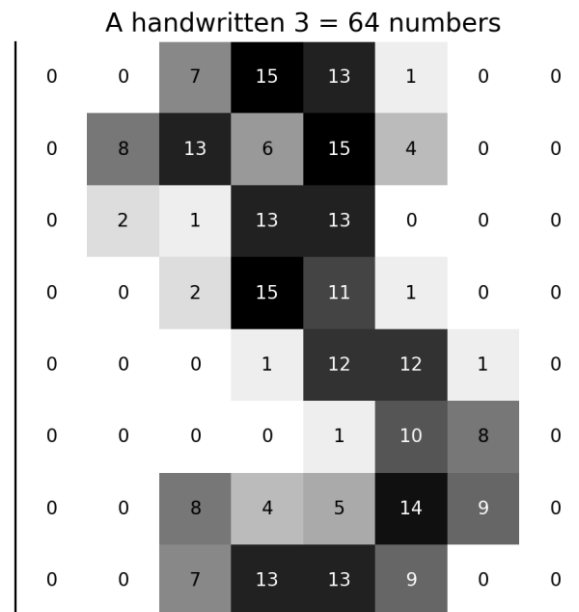


Figure: to a computer, a handwritten 3 is a grid of 64 numbers (0 = white, 16 = black).

Explanation

To a computer, the handwritten 3 is not a shape — it is 64 numbers, one for each pixel, where 0 is white and 16 is black. Writing a rule means writing logic like “if pixel 13 is greater than 10 and pixel 21 is less than 3 and...”. With 64 inputs, each taking 17 possible values, the number of possible images is astronomically large.

For a real photograph the situation is far worse: a small 256 by 256 colour image already contains almost two hundred thousand numbers. No human can write explicit rules over that many inputs.

This is not only true for images. Think of spam e-mails, fraud detection, speech recognition or predicting house prices: the patterns are real but too complex, too numerous, or change too quickly for us to describe by hand. Machine learning solves this by finding the rules automatically from examples.

Traditional programming vs machine learning

Slide 12

Key points

- Traditional: Data + Rules → Program → Answers
- Machine learning: Data + Answers → Learning algorithm → Rules (a model)
- The learned model is then used to make predictions on new data

Explanation

This diagram is the single most important picture of today's lecture. In traditional programming, a human writes the rules. We feed data and rules into the computer and it produces answers. For example, a tax calculator: the rules are the tax law, the data is your income, the answer is the tax you owe.

In machine learning we flip this around. We feed the computer data together with the answers we want — for example thousands of images each labelled with the correct digit — and the computer produces the rules. We call that learned rule a model. Once we have the model, we can use it like a traditional program: give it new data and it produces answers, which we call predictions.

So machine learning is not magic; it is a different way of creating programs. The programmer's job changes from writing rules to collecting good data, choosing a learning method and checking that the learned rules work.

A classic definition (Arthur Samuel, 1959)

Slide 13

Key points

- ML: giving computers the ability to learn without being explicitly programmed (Samuel, commonly paraphrased)
- Samuel's checkers program improved by playing against itself
- Intuitive but informal definition

Explanation

The term “machine learning” is usually credited to Arthur Samuel, an IBM researcher who in the 1950s wrote a program that played checkers. Samuel was not a great checkers player himself, so he could not simply write down winning rules. Instead his program played many games against itself, remembered which board positions tended to lead to wins, and gradually became a strong player.

His idea is usually summarised as: machine learning is the field of study that gives computers the ability to learn without being explicitly programmed. Note the key phrase “without being explicitly programmed”. Of course we still write code — but we write code that learns, not code that contains the answer.

This definition is intuitive but informal. It tells us the spirit of ML, but not how to check whether a program is actually learning. For that we need a more precise definition, which comes next.

A precise definition: Task, Experience, Performance

Slide 14

Key points

- Mitchell (1997): a program learns if its performance P at task T improves with experience E
- T = task (what to do); E = experience (the data); P = performance measure (how well)
- Defining T , E and P is the first step of every ML project

Explanation

Tom Mitchell gave a more precise, engineering-style definition in his 1997 textbook. In plain words: a program learns if its performance on some task, measured in some way, gets better as it gains experience.

So whenever someone says “this system uses machine learning”, you can ask three questions. What is the task T — what is the system actually trying to do? What is the experience E — what data does it learn from? And what is the performance measure P — how do we know whether it is doing well?

If you cannot answer all three, the problem is not yet well defined. This is exactly what you will do at the start of your midterm and capstone projects: define T, E and P before writing any model code. It is also the first criterion in the project rubric — “problem understanding”.

Applying T, E and P to real problems

Slide 15

Key points

- Go: choose moves / games played / % games won
- House prices: predict price / past sales / average error (MAE)
- Medical X-ray: flag disease / labelled X-rays / recall
- Customer groups: find groups / purchase history / cluster quality

Explanation

Let's practise the T-E-P framework on four examples. For the game of Go, the task is choosing moves, the experience is games played, including games the system plays against itself, and performance is the percentage of games won. For house prices, the task is predicting a price, the experience is records of past sales, and the performance is how far predictions are from real prices — for example the average error in dollars, which we will call MAE in Week 4.

For medical imaging, the task might be flagging chest X-rays that show signs of pneumonia; experience is X-rays labelled by radiologists; performance could be recall — the proportion of real cases the model catches — because missing a sick patient is costly. For customer grouping, there are no labels at all: we simply want sensible groups, and measuring performance is harder.

Notice that the performance measure depends on what matters in the real problem. Choosing it is a design decision, not a detail.

AI, machine learning and deep learning

Slide 16

Key points

- AI: any technique that makes computers behave intelligently (incl. rule-based systems)
- ML: a subset of AI that learns behaviour from data
- Deep learning: a subset of ML using many-layered neural networks
- This course focuses on classical ML — ideal for small, tabular datasets

Explanation

You will hear the terms AI, machine learning and deep learning used interchangeably in the media, but they are not the same. Think of them as nested circles.

Artificial intelligence is the broadest field: any technique that makes computers behave in ways we would call intelligent. That includes old-style expert systems built from hand-written rules, search algorithms like those used in chess engines, and planning systems.

Machine learning is a subset of AI: systems that learn their behaviour from data instead of from hand-written rules. Everything in this course — regression, k-NN, decision trees, Naive Bayes, k-means — is classical machine learning.

Deep learning is a subset of machine learning that uses artificial neural networks with many layers. It powers modern image recognition, speech recognition and large language models such as ChatGPT. Deep learning needs a lot of data and computing power; classical ML often works very well on smaller, tabular datasets — which is what you will typically have in your projects.

ML, statistics and data science: close relatives

Slide 17

Key points

- Statistics: inference — understand relationships and uncertainty
- Machine learning: prediction — perform well on unseen data
- Data science: the full process from data to decisions, using both
- Your statistics background is directly useful in ML

Explanation

Machine learning shares a lot with statistics and data science, and you will see overlapping ideas throughout the course — linear regression, for example, is a classic statistical method and also a basic ML model.

The difference is mainly one of emphasis. Statistics focuses on inference: understanding relationships, testing hypotheses and quantifying uncertainty — for example, is this drug effective, and how sure are we? Machine learning focuses on prediction: making accurate predictions on new, unseen data, often with complex models, even if we cannot fully explain every parameter. Data science is the broader practice of turning data into decisions, and it uses both statistics and ML, plus data engineering, visualisation and communication.

For you, the practical message is: your probability and statistics course is directly useful here. Means, variances, distributions and correlation will appear again and again.

Why is machine learning booming now?

Slide 18

Key points

- Data: web, smartphones, sensors and digital records
- Compute: GPUs and cloud computing (e.g. free Colab)
- Algorithms: better training methods, especially for deep networks
- Open source: scikit-learn, PyTorch, TensorFlow, Kaggle datasets

Explanation

Many ML ideas are decades old — linear regression goes back to the early 1800s, and neural networks to the 1950s. So why has ML exploded only in the last fifteen years? Four forces came together.

First, data. Smartphones, the web, sensors and digital records produce enormous amounts of data, and ML methods improve with more examples. Second, computing power. Graphics processing units, originally built for video games, turned out to be ideal for the matrix calculations inside ML, and cloud computing made that power available to anyone — including you, for free, in Google Colab. Third, better algorithms and techniques, especially for training deep neural networks. And fourth, open-source software and open

datasets: libraries like scikit-learn, PyTorch and TensorFlow, and platforms like Kaggle, mean a student today can do in ten lines of code what required a research team twenty years ago.

A brief history of machine learning

Slide 19

Key points

- 1950 Turing: “Can machines think?”
- 1957 Rosenblatt's perceptron
- 1959 Samuel's checkers program; term “machine learning”
- 1986 Backpropagation popularised
- 1997 Deep Blue beats Kasparov
- 2012 AlexNet starts the deep-learning era
- 2016 AlphaGo beats Lee Sedol
- 2022 ChatGPT

Explanation

Let's take a quick tour through history. In 1950 Alan Turing asked “Can machines think?” and proposed the imitation game, now called the Turing test. In 1957 Frank Rosenblatt built the perceptron, an early artificial neuron that could learn simple patterns. In 1959 Arthur Samuel popularised the term machine learning with his checkers program.

In 1986 Rumelhart, Hinton and Williams popularised backpropagation, the algorithm that made training multi-layer neural networks practical. In 1997 IBM's Deep Blue defeated world chess champion Garry Kasparov — though mostly using search, not learning. In 2012 a deep neural network called AlexNet won the ImageNet image-recognition challenge by a large margin and started the deep-learning era. In 2016 DeepMind's AlphaGo defeated Lee Sedol at Go, a game long considered too complex for computers. And in late 2022, ChatGPT brought large language models to hundreds of millions of people.

Notice the pattern: long periods of slow progress, sometimes called “AI winters”, followed by sudden breakthroughs when data, compute and algorithms came together.

Part 2 — How machines learn

Data, features, labels, models — and the training loop

Now we open the black box. In this part we will learn the vocabulary of machine learning — samples, features, labels, models and parameters — and see how a model actually learns by reducing its error, including an animation of a model learning in real time. We will finish with the most important idea in ML: generalisation, and the danger of overfitting.

Anatomy of a dataset

Slide 21

Key points

- Row = sample / instance / observation
- Feature columns = inputs, written X (numerical or categorical)
- Label / target column = what we predict, written y
- Labelled data → supervised learning; no labels → unsupervised

Explanation

Almost every dataset you will use in this course is a table. Each row is one example — also called a sample, an instance or an observation. Here, each row is one house.

Each column that describes the example is a feature — also called an input variable, attribute or predictor. Here the features are size, number of rooms, age of the house and district. Some features are numerical, like size and age; others are categorical, like the district. In Week 3 we will learn how to convert categorical features into numbers.

The column we want to predict is the label or target, usually written y. Here it is the price. The features are usually written as X — a capital letter because it is a whole table, or matrix. In scikit-learn and pandas you will literally write X = the feature columns and y = the target column. When the label is known for every row, we can do supervised learning; when there is no label column, we are in unsupervised learning.

Key vocabulary you will use every week

Slide 22

Key points

- Model: the learned rule mapping inputs to outputs
- Parameters: learned from data (e.g. slope, intercept)
- Hyperparameters: chosen by us before training (e.g. k in k-NN)
- Training vs prediction (inference)
- Loss / error and generalisation

Explanation

Here is the core vocabulary of machine learning in one table. Please copy it into your notes; we will use these words in every lecture and lab.

A few points deserve emphasis. A model is the learned rule — the thing that maps inputs to outputs. Parameters are the numbers inside the model that are learned from data, such as the slope and intercept of a line. Hyperparameters are settings that we, the humans, choose before training — like the number of neighbours k in k -NN or the maximum depth of a decision tree. We will tune hyperparameters from Week 6 onwards.

Training means fitting the parameters to data; prediction or inference means using the trained model on new inputs. Loss, or error, measures how wrong the model is, and learning is essentially the process of reducing that loss. Finally, generalisation is how well the model performs on new, unseen data — which is what we really care about.

A model is a function with adjustable knobs

Slide 23

Key points

- Model: $\hat{y} = f(x; \theta)$ — a function with parameters θ (the knobs)
- Example: $\hat{y} = w \cdot x + b$ (a straight line)
- Error = $\hat{y} - y$; loss combines errors over all examples
- Learning = find the parameters that minimise the loss

Explanation

Mathematically, a model is just a function. It takes the features x as input and produces a prediction, written $\hat{y}$ — the little hat means “estimated”. Inside the function are parameters, usually written with the Greek letter θ or as w and b . You can think of the parameters as knobs.

For example, the simplest regression model is a straight line: $\hat{y}$ equals w times x plus b . With one setting of the knobs the line is steep; with another it is flat. Learning means turning the knobs until the predictions $\hat{y}$ are as close as possible to the true labels y for the training examples.

The difference between $\hat{y}$ and y is the error. We combine the errors over all training examples into a single number called the loss. So the whole learning process can be summarised in one sentence: find the parameter values that make the loss as small as possible. Different ML algorithms differ mainly in what function they use and how they search for good parameters.

Learning = reducing error

Slide 24

Key points

- Residual = true value – predicted value
- MSE = average of squared residuals
- Training finds the w and b that minimise MSE
- The trained line predicts prices for new houses

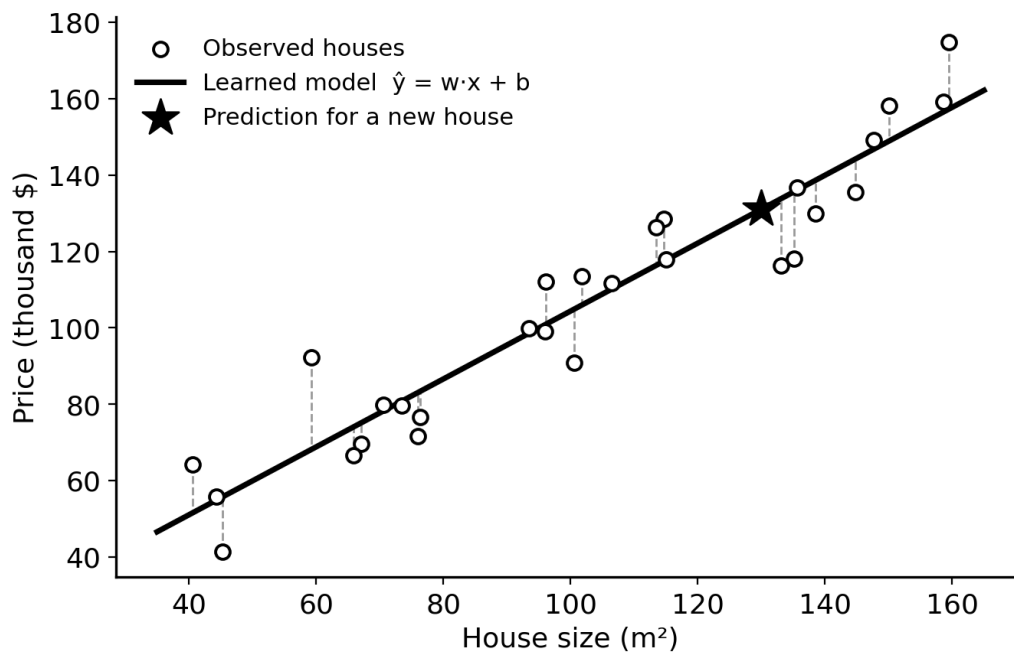


Figure: a linear regression line fitted to synthetic house data; dashed lines are residuals.

Explanation

Here is a concrete example. Each circle is a house: its size on the horizontal axis and its price on the vertical axis. The black line is a linear regression model learned from these points. The dashed vertical lines are the errors, also called residuals: the gap between the price the model predicts and the true price.

To measure the overall error we usually square each residual and take the average. This is the mean squared error, MSE. We square the errors for two reasons: so that positive and negative errors don't cancel out, and so that big errors are punished more than small ones.

Training chooses the slope w and the intercept b that give the smallest possible MSE. Once trained, we can use the line to predict the price of a new house we have never seen — the black star. You will compute MSE, its square root RMSE, and the mean absolute error MAE yourselves in the Week 4 lab.

Video: watch a model learn

Slide 25

Key points

- Animation: gradient descent fitting a line
- Each step nudges w and b to reduce the error
- Error drops quickly, then levels off
- Large neural networks learn in essentially the same way

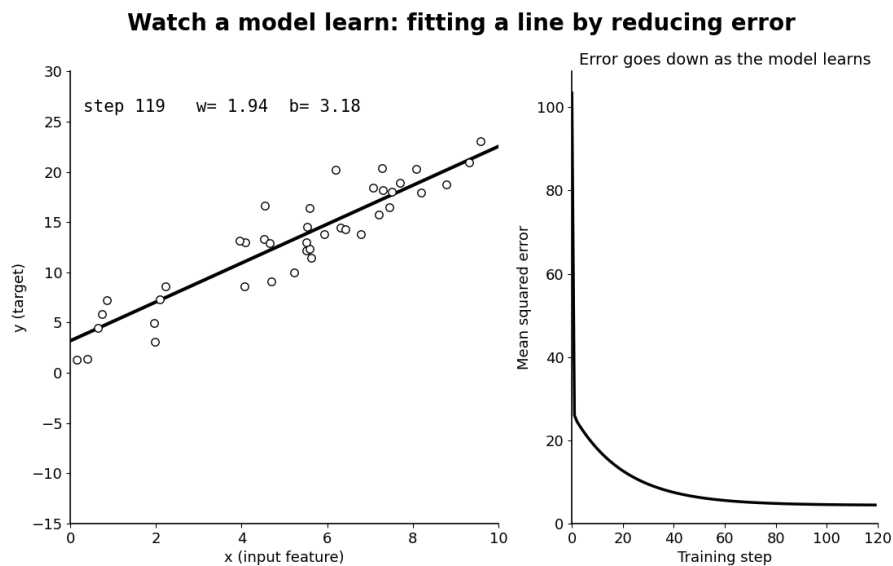


Figure: final frame of the gradient-descent animation shown in class.

Explanation

Now let's watch learning happen. Click the video to play it. On the left you see data points and a line. At the start the line has random parameters and is completely wrong — it even slopes downwards. On the right you see the mean squared error.

At each training step, the algorithm calculates in which direction to nudge the slope and intercept so that the error decreases, and it moves them a small amount in that direction. This method is called gradient descent. Watch how the line rotates and moves until it fits the cloud of points, and how the error curve drops quickly at first and then flattens out.

This is literally all “learning” means for this model: repeated small adjustments of parameters to reduce error. Much bigger models, including the neural networks behind modern AI, learn in essentially the same way — just with millions or billions of parameters instead of two.

The training loop

Slide 26

Key points

- 1. Initialise parameters
- 2. Predict on training data
- 3. Measure the loss
- 4. Update parameters to reduce the loss
- Repeat until the loss stops improving — hidden inside `model.fit(X, y)`

Explanation

The animation showed a training loop, and almost every ML algorithm follows the same loop in some form. Step one: start with some initial parameters — often random. Step two: use the current model to make predictions on the training data. Step three: compare the predictions with the true labels and measure the loss. Step four: adjust the parameters in a direction that reduces the loss. Then repeat steps two to four many times, until the loss stops improving.

Some algorithms, like linear regression, can actually jump straight to the best parameters using a formula. Others, like decision trees, build the model step by step with greedy choices. And k-nearest neighbours doesn't really train at all — it just memorises the data. But the idea of measuring error and improving is common to all of them.

In scikit-learn, this entire loop is hidden inside one line: `model.fit(X, y)`. That's convenient, but it's important you understand what's happening inside that line.

Training data vs test data

Slide 27

Key points

- Never evaluate on the data used for training
- Typical split: ~70–80% train, ~20–30% test
- Test set = a fair exam with unseen questions
- Data leakage makes test scores over-optimistic (Week 3)

Explanation

Here is one of the most important rules in machine learning: never evaluate a model on the same data you trained it on. Think of an exam. If the teacher gives you the exact exam questions to practise beforehand, a high score only proves you memorised the answers, not that you understood the subject. A fair exam uses new questions.

In ML we do the same. Before training, we split the dataset: typically around 70 to 80 percent for training and 20 to 30 percent for testing. The model learns only from the training set. The test set is locked away and used only at the end to estimate how the model will perform on truly new data.

If information from the test set leaks into training — even indirectly, for example by computing a scaling factor on the whole dataset — the test score becomes over-optimistic. This is called data leakage, and we will study it in detail in the Week 3 lab. In code, the split is one line: `train_test_split` from scikit-learn.

Generalisation: underfitting vs overfitting

Slide 28

Key points

- Underfitting (high bias): model too simple, misses the pattern
- Overfitting (high variance): model memorises noise, fails on new data
- Goal: the right complexity that generalises
- Week 8: controlling overfitting in decision trees

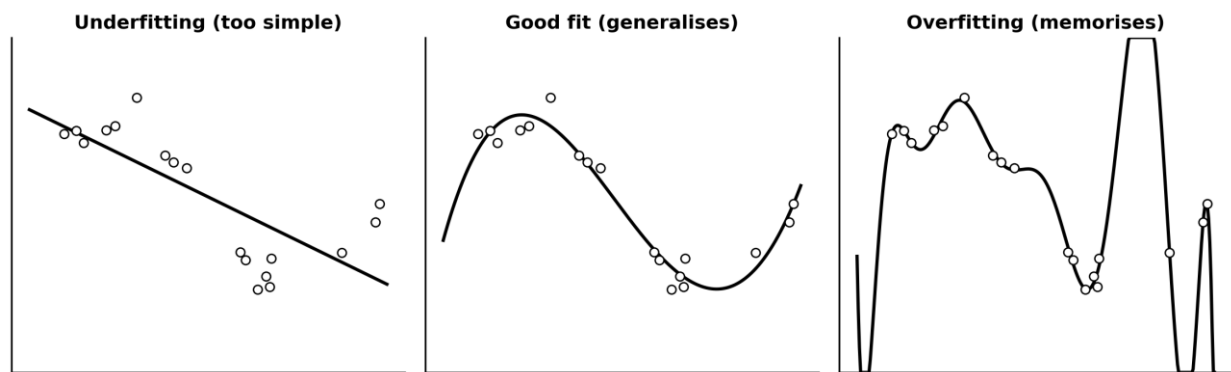


Figure: the same data fitted by models that are too simple, about right, and too complex.

Explanation

The goal of machine learning is not to fit the training data perfectly; it is to generalise — to make good predictions on new data. These three plots show the same noisy data fitted by three models of different complexity.

On the left, a straight line is too simple for this curved pattern. It makes large errors even on the training data. This is underfitting, also called high bias. On the right, a very flexible polynomial passes close to every single training point, but between the points it swings wildly. It has memorised the noise rather than learned the pattern. This is overfitting, also called high variance. It will perform badly on new data. In the middle, a model of moderate complexity captures the real shape without chasing the noise. That is the one we want.

A useful analogy: the underfitting student didn't study; the overfitting student memorised last year's answers word for word; the good student understood the ideas. You will fight overfitting directly in Week 8 when we limit the depth of decision trees.

The complexity trade-off

Slide 29

Key points

- Training error always falls as complexity grows
- Test error falls, then rises (overfitting)
- Sweet spot = lowest test error
- Large train–test gap = warning sign of overfitting

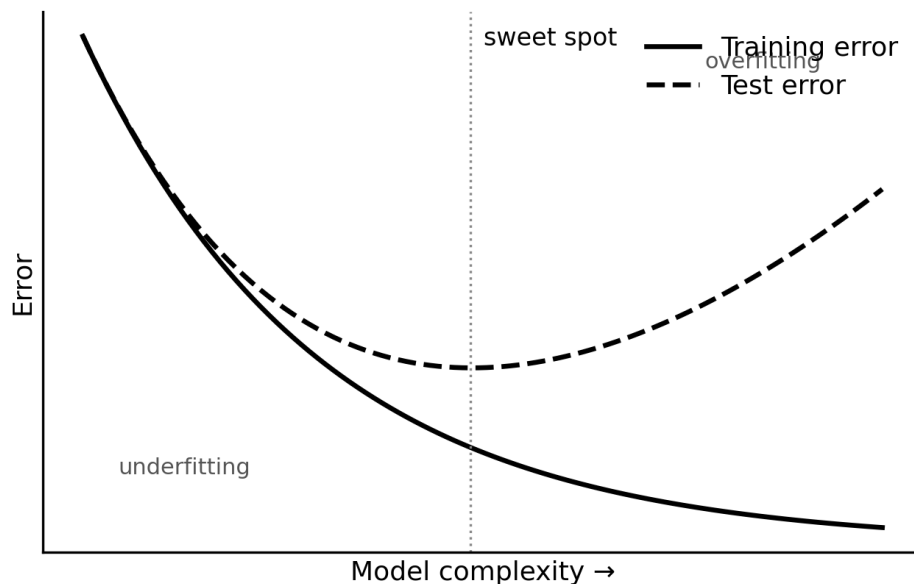


Figure: typical training and test error as model complexity increases (illustrative).

Explanation

This graph summarises the previous slide. On the horizontal axis is model complexity — for example, the degree of the polynomial or the depth of a decision tree. The solid line is the error on the training data, and the dashed line is the error on the test data.

As complexity increases, training error keeps going down: a flexible enough model can always fit the training data better. But the test error behaves differently. At first it also goes down, because the model is learning real structure. After a certain point, it starts to rise again, because the model is now fitting noise. The sweet spot is where test error is lowest.

The practical lesson: a big gap between a very low training error and a much higher test error is the classic warning sign of overfitting. Always report both. When you tune hyperparameters like k in k -NN or tree depth later in the course, you are really searching for this sweet spot.

More (good) data usually helps

Slide 30

Key points

- Test accuracy rises with more training data, then levels off
- More data reduces overfitting to noise
- Quality > quantity: wrong labels teach wrong rules
- Data must represent the real population

Explanation

One of the most reliable ways to improve a model is to give it more good-quality training data. This chart is illustrative, but it shows the typical shape of a learning curve: as the number of training examples grows, test accuracy improves quickly at first and then levels off.

Why? With few examples, the model can easily be fooled by noise and random quirks of the sample — it overfits. With more examples, the real pattern becomes clearer and the noise averages out. But the curve

flattens: after a certain point, extra data brings only small gains, and the limit is set by the model and the quality of the features.

Two important warnings. First, quality matters more than quantity: a million wrongly labelled examples teach the wrong rule. Second, data must be representative: if your training data only contains patients from one hospital, the model may fail on patients from another. We will return to these ideas when we discuss limitations and bias.

Part 3 — Types of machine learning

Supervised, unsupervised and reinforcement learning

In this part we meet the three main families of machine learning. The difference between them is the kind of feedback the learner gets: correct answers, no answers at all, or rewards and penalties. Knowing which family your problem belongs to is the first step in choosing a model — and it is exactly learning outcome 2 of this course.

The three main types of machine learning

Slide 32

Key points

- Supervised (labelled data): regression (number) and classification (category)
- Unsupervised (no labels): clustering, dimensionality reduction, anomaly detection
- Reinforcement (rewards): agent learns by trial and error
- This course: mostly supervised, plus k-means clustering

Explanation

This tree is your map of machine learning. At the top level there are three families.

Supervised learning learns from labelled examples: every training example comes with the correct answer. It splits into regression, where the answer is a number such as a price or a temperature, and classification, where the answer is a category such as spam or not spam. Most of this course — weeks 4 to 10 — is about supervised learning.

Unsupervised learning works with unlabelled data. There is no correct answer; instead the goal is to discover structure. The main tasks are clustering — grouping similar items, which we study with k-means in Week 11 — dimensionality reduction, and anomaly detection.

Reinforcement learning is different again: an agent learns by acting in an environment and receiving rewards or penalties. It is behind game-playing systems like AlphaGo and is used in robotics. We will only introduce it today; it is beyond the scope of this course.

Supervised learning: learning with a teacher

Slide 33

Key points

- Training data = input–output pairs (the correct answers act as a teacher)
- Learns a mapping from X to y , then predicts y for new X
- Labels can be expensive (experts) or free (recorded outcomes)
- Project tip: check that your dataset has a clear label column

Explanation

Supervised learning is called “supervised” because the training data acts like a teacher who provides the correct answer for every example. The model sees many pairs of inputs and correct outputs, and learns a function that maps one to the other. Afterwards, it can produce outputs for new inputs where the answer is unknown.

Think of learning a language with flashcards: the front shows a word, the back shows the translation. After enough cards, you can translate words you've seen in new sentences.

The main cost of supervised learning is labels. Someone has to provide the correct answers, and that can be expensive: radiologists labelling X-rays, lawyers labelling documents, or thousands of people tagging images. Sometimes labels come for free — for example, the actual sale price of a house, or whether a customer clicked on an ad. When you choose a project dataset, check first: does it have a clear label column?

Regression vs classification

Slide 34

Key points

- Regression: target is a number (price, temperature, energy use)
- Classification: target is a category (spam/not spam, digit 0–9)
- Regression metrics: MAE, MSE, RMSE (Week 4)
- Classification metrics: accuracy, precision, recall, F1, confusion matrix (Week 5)
- Tip: “Could the answer be 3.7?” → regression

Explanation

Supervised learning has two main flavours, and the difference is simply the type of the target variable.

In regression, the target is a continuous number. How much will this house sell for? What will tomorrow's temperature be? How many kilowatt-hours will this building use next month? The model outputs a number, and we evaluate it by how far off it is — using MAE, MSE or RMSE, which you will compute in Week 4.

In classification, the target is a category or class. Is this e-mail spam or not? Which digit is in this image? Will this customer cancel their subscription — yes or no? The model outputs a class label, often together with a probability. We evaluate it with accuracy, precision, recall, F1-score and the confusion matrix, in Week 5.

A quick test: ask yourself “Could the answer be 3.7?” If yes, it's usually regression. If the answer must be one of a fixed list of options, it's classification. Careful: a name like “logistic regression” is misleading — it is actually a classification method.

Classification: learning a decision boundary

Slide 35

Key points

- Each e-mail = a point in feature space
- The classifier learns a decision boundary separating classes
- New points are classified by which side they fall on
- Different models draw different boundary shapes (linear vs flexible)

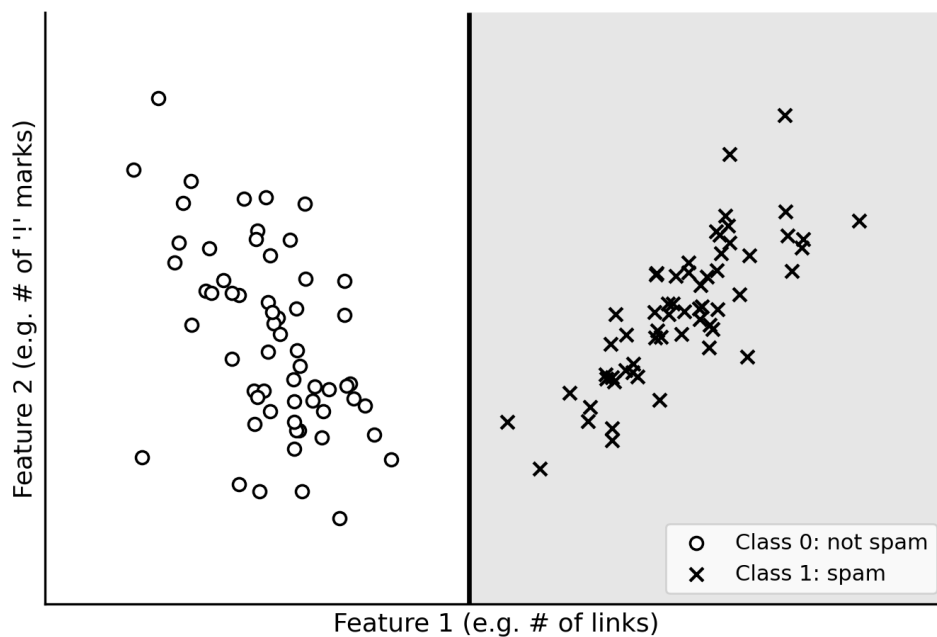


Figure: a linear decision boundary separating two classes (synthetic data).

Explanation

Here is what a classifier looks like geometrically. Each point is an e-mail described by two features — for example, the number of links it contains and the number of exclamation marks. Circles are genuine e-mails, crosses are spam.

The classifier learns a decision boundary: the black line. Everything on one side (the grey region) is predicted as spam; everything on the other side is predicted as not spam. When a new e-mail arrives, we compute its features, see which side of the line it falls on, and that is the prediction.

Real problems have many more than two features — a spam filter might use thousands, one per word — so the boundary lives in a high-dimensional space we cannot draw. But the idea is exactly the same. Different classifiers draw different shapes of boundary: logistic regression draws straight lines, k-nearest neighbours and decision trees can draw much more flexible, wiggly boundaries. That flexibility is useful, but it also brings the risk of overfitting we saw earlier.

Video: a classifier learning its boundary

Slide 36

Key points

- Animation: logistic regression trained with gradient descent
- The boundary shifts and rotates to reduce errors
- Accuracy climbs as training proceeds
- A few errors remain — perfect training accuracy often means overfitting

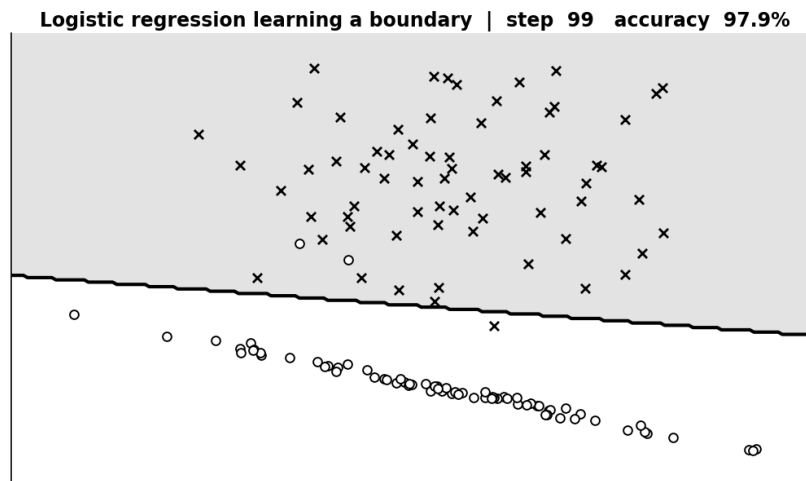


Figure: final frame of the logistic-regression animation shown in class.

Explanation

Click to play the second animation. This is logistic regression — the classifier you will study in Week 5 — being trained with gradient descent on a two-class dataset.

At the start the boundary is in a poor position and many points are on the wrong side, so the accuracy shown in the title is low. At every step, the algorithm looks at the points it gets wrong or is unsure about, and shifts and rotates the boundary to reduce the error. Within a few dozen steps the boundary settles into a position that separates the circles and crosses almost perfectly, and the accuracy climbs towards 98 percent.

Pay attention to the fact that the final boundary is not perfect: a few points are still on the wrong side. That is normal and even healthy. Real data overlaps, and a model that insisted on classifying every single training point correctly would probably be overfitting.

Unsupervised learning: finding hidden structure

Slide 37

Key points

- No labels — only features
- Goal: discover structure, e.g. groups of similar items (clustering)
- Example: customer segmentation for marketing
- Challenge: evaluation without correct answers (Week 11)

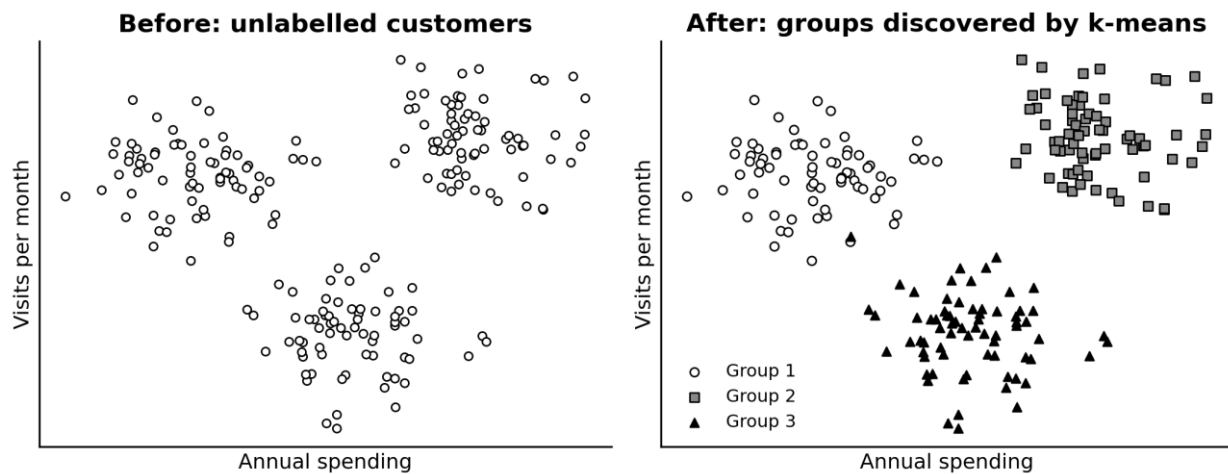


Figure: unlabelled customers (left) and the groups found by k-means (right).

Explanation

In unsupervised learning there are no labels. We only have the features, and the goal is to discover structure hidden in the data.

The most common unsupervised task is clustering: grouping similar data points together. On the left you see customers of a shop described by two features: how much they spend per year and how often they visit. There is no label telling us which “type” each customer is. On the right, the k-means algorithm has discovered three natural groups — perhaps occasional low spenders, frequent moderate spenders and loyal high spenders. The shop could then design a different marketing strategy for each group.

The big challenge in unsupervised learning is evaluation. With no correct answers, how do we know the groups are good? We use a mix of numeric measures and human judgement: do the clusters make sense, and are they useful? You will explore this in Week 11, including how to choose the number of clusters k .

Video: k-means clustering step by step

Slide 38

Key points

- Start with k centres (★)
- Assign: each point joins its nearest centre
- Update: each centre moves to the mean of its points
- Repeat until the centres stop moving (convergence)

k-means clustering (k = 4) | iteration 8 | move each ★ to its group's centre

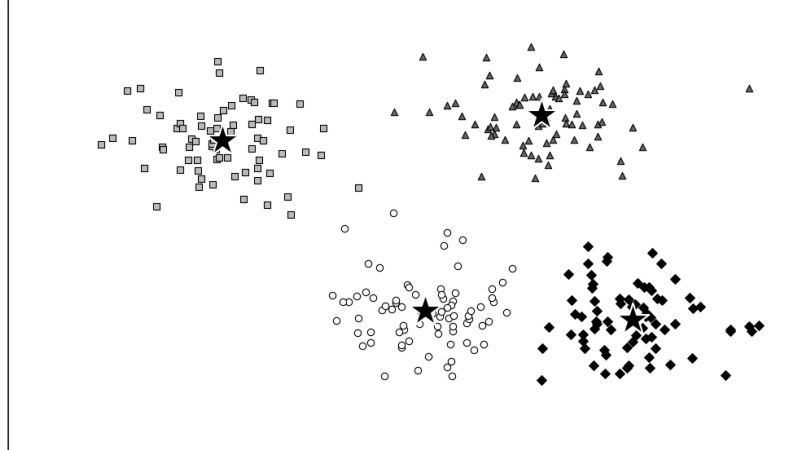


Figure: final frame of the k-means animation shown in class.

Explanation

The third animation shows how k-means works. We ask for four clusters, so we start with four centres, shown as black stars, placed in arbitrary positions.

The algorithm then repeats two simple steps. Step one, assign: every point joins the cluster of its nearest star — watch the marker shapes change. Step two, update: each star moves to the average position, the centre, of the points assigned to it. Then we assign again, update again, and so on.

After a few iterations, the stars stop moving: the clusters are stable and the algorithm has converged. That is the entire algorithm — simple enough to implement yourself in about fifteen lines of Python, which is a nice optional challenge before Week 11. One important note: the result can depend on where the stars start, which is why in practice k-means is run several times with different starting positions.

Other unsupervised tasks

Slide 39

Key points

- Anomaly detection: learn what is normal, flag the unusual (fraud, faults, attacks)
- Dimensionality reduction: compress many features into a few (e.g. PCA)
- Used for visualisation, compression and speed
- Iris: 4 measurements → 2 components reveal the species structure

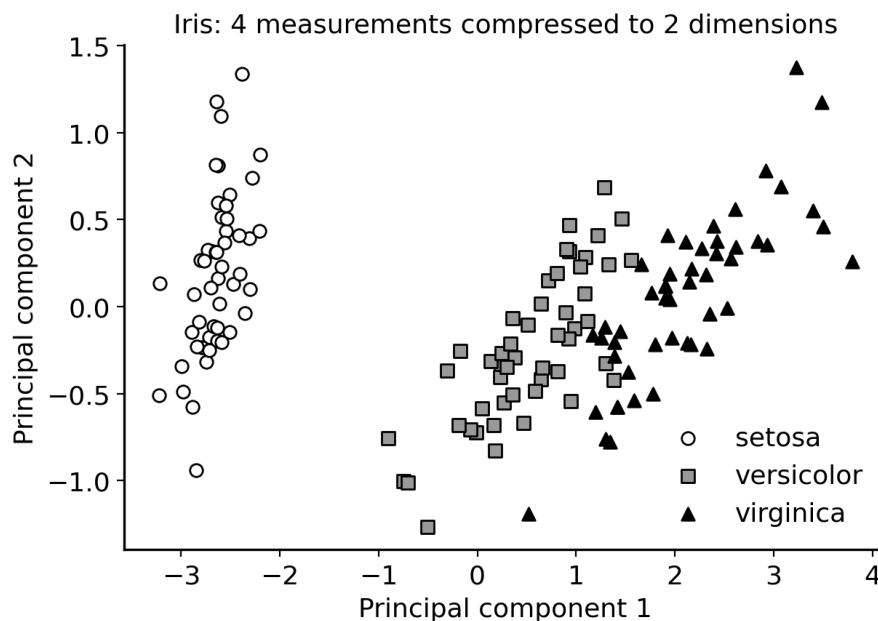


Figure: PCA compresses the four Iris measurements into two dimensions.

Explanation

Clustering is not the only unsupervised task. Here are two more you should know about.

On the left is anomaly detection: learning what “normal” looks like and flagging points that don't fit. Most transactions on a bank card follow regular patterns; a few, far outside the normal cloud, might be fraud. Anomaly detection is also used to spot failing machines in factories from sensor data, or unusual network traffic that could indicate a cyber-attack.

On the right is dimensionality reduction. The famous Iris dataset has four measurements for each flower. We cannot plot four dimensions, but a technique called principal component analysis, PCA, compresses the four measurements into two new ones that keep most of the information. Now we can see the data — and notice that the three species form clearly separated groups, even though PCA never saw the species labels. Dimensionality reduction is used for visualisation, compression and speeding up other algorithms.

Reinforcement learning: learning by trial and error

Slide 40

Key points

- Agent observes state → takes action → receives reward and new state
- Goal: learn a policy that maximises total reward
- Analogy: training a dog with treats
- Uses: games (AlphaGo), robotics, control; beyond this course's scope

Explanation

The third family is reinforcement learning. Here there is no fixed dataset at all. Instead, an agent interacts with an environment. At each step the agent observes the current state, chooses an action, and the environment responds with a new state and a reward — a number that says how good the outcome was. The agent's goal is to learn a strategy, called a policy, that maximises the total reward over time.

Think of training a dog: you don't show it labelled examples of sitting; you reward it when it sits. Or think of learning to ride a bicycle — you learn from falling and not falling.

Reinforcement learning is powerful for sequential decision problems: games such as Go and chess, robot control, and managing resources such as data-centre cooling. It is also used to fine-tune large language models with human feedback. However, it usually needs a huge number of trials, often in simulation. It is beyond the scope of this course, but it's important to know it exists.

Beyond the big three

Slide 41

Key points

- Semi-supervised: few labels + many unlabelled examples
- Self-supervised: labels created from the data itself (how LLMs are pre-trained)
- Transfer learning: reuse a model trained on a big task for a new, smaller task

Explanation

The three families are a useful map, but modern ML also uses several hybrid approaches. You don't need to master these now, but you will hear the terms.

Semi-supervised learning uses a small amount of labelled data together with a large amount of unlabelled data. This is common when labels are expensive — for example, a hospital might have a million scans but only a thousand labelled by a specialist.

Self-supervised learning creates labels automatically from the data itself. For example, hide a word in a sentence and train the model to guess it, or hide part of an image and ask the model to fill it in. This is how large language models are pre-trained on enormous amounts of text without human labelling.

Transfer learning takes a model trained on one large task and adapts it to a new, smaller task. For example, a model trained on millions of general photos can be fine-tuned to recognise plant diseases from just a few hundred leaf images. This is extremely practical, and it is how most real-world deep learning projects start today.

Quick quiz: which type of ML?

Slide 42

Key points

- A: next month's electricity use
- B: skin lesion benign or malignant
- C: group news articles into topics
- D: robot arm learns to grasp from rewards
- E: predict a student's exam score
- F: flag unusual logins (no labels)

Explanation

Time for a short quiz. For each scenario, decide: is it supervised regression, supervised classification, unsupervised learning, or reinforcement learning? Work with your neighbour for two minutes and write down your answers. I'll reveal the answers on the next slide.

Scenario A: predict how many kilowatt-hours a building will use next month, using records of past months.

Scenario B: detect whether a skin lesion in a photo is benign or malignant, using photos labelled by

dermatologists. Scenario C: group news articles into topics when no topics are given. Scenario D: teach a robot arm to pick up objects by rewarding successful grasps. Scenario E: predict a student's final exam score from attendance and homework marks. Scenario F: flag unusual login attempts to a university system when no attack examples are labelled.

Hint: look first for the label. Is there one? Is it a number or a category? Or is there a reward instead?

Quiz answers

Slide 43

Key points

- A: regression (number)
- B: classification (two classes; recall matters)
- C: unsupervised clustering
- D: reinforcement learning
- E: regression (or classification if framed as pass/fail)
- F: unsupervised anomaly detection

Explanation

Let's check the answers. A is regression: the target, energy use in kilowatt-hours, is a number, and we have past values as labels. B is classification: two categories, benign or malignant, with expert labels. This is a real application area, and a place where the choice of metric matters a lot — missing a malignant lesion is much worse than a false alarm, so recall is critical.

C is unsupervised — specifically clustering, because there are no topic labels. D is reinforcement learning: no labelled examples, only rewards for successful actions. E is regression again: exam score is a number. Some of you may have argued for classification if the goal were pass or fail — and that's a good point: the same data can be framed as either type depending on the question you ask. F is unsupervised anomaly detection: we learn what normal log-ins look like and flag unusual ones.

If you got at least four right, you already understand the most important decision in ML: framing the problem.

Part 4 — ML in the real world

Where machine learning is used today — and where you could use it

Now let's leave theory for a while and look at how machine learning is used in practice. We'll survey major application areas, then look more closely at four cases: spam filtering, healthcare, recommender systems and computer vision. We will finish with ideas for projects that you could realistically complete in this course.

Machine learning is everywhere

Slide 45

Key points

- Healthcare: diagnosis support, risk prediction, drug discovery
- Finance: fraud detection, credit scoring
- Retail: recommendations, demand forecasting
- Transport: arrival-time estimates, route optimisation, driver assistance
- Language: translation, speech recognition, chatbots
- Vision: face unlock, quality inspection, crop monitoring

Explanation

Machine learning is already part of your daily life, often invisibly. Here are six major areas.

In healthcare, ML supports diagnosis from medical images and signals such as ECGs, predicts which patients are at risk of readmission, and speeds up drug discovery. In finance, it detects fraudulent card transactions within milliseconds, scores credit risk and automates document processing. In retail and online services, it powers product recommendations, demand forecasting and dynamic pricing.

In transport, it estimates arrival times in map and taxi apps, optimises delivery routes and is a key part of driver-assistance systems. In language, it drives machine translation, speech recognition, chatbots and search engines. And in vision, it enables face unlock on your phone, quality inspection in factories and crop monitoring from satellite or drone images.

As you use your phone this week, try to spot at least three places where machine learning is working in the background — it is a good way to start thinking like an ML engineer.

Case study 1: the spam filter

Slide 46

Key points

- E-mail → features (word counts, links, sender) → classifier → spam probability → folder
- Naive Bayes is a classic spam classifier (Week 9)
- Spammers adapt; ML filters can be retrained on new examples
- Clicking “report spam” creates new labels

Explanation

Spam filtering is the classic first example of machine learning, and it neatly shows the whole pipeline. An e-mail arrives. First we turn it into features — numbers the model can use. For text, a simple approach is to count how often each word appears: “free”, “winner”, “invoice”, and so on. We can also add other features: the number of links, whether the sender is in your contacts, the time it was sent.

These features go into a trained classifier — historically, Naive Bayes was the famous choice, and you will build a small text classifier yourself in Week 9. The classifier outputs a probability that the e-mail is spam. If it is above a threshold, say 0.9, the e-mail goes to the spam folder.

Why is ML better than rules here? Because spammers adapt. Every time you write a rule, they change their wording. An ML filter can be retrained regularly on the latest examples, and every time you click “report spam”, you are providing a new label — you are part of the training process.

Case study 2: machine learning in healthcare

Slide 47

Key points

- Inputs: medical images, biosignals (ECG, EEG), health records
- Tasks: detect disease, predict risk, prioritise cases
- Risks: data shift between hospitals, costly errors, trust
- ML as decision support — the clinician decides

Explanation

Healthcare is one of the most promising and most sensitive areas for machine learning, and it is close to my own background in biomedical engineering.

Machine learning can analyse medical images such as X-rays, retinal scans and skin photos; biosignals such as ECG and EEG; and electronic health records to predict risks such as hospital readmission. Research systems have reached specialist-level performance on some narrow image tasks, such as detecting diabetic eye disease from retinal photographs.

But healthcare also shows clearly why ML must be used responsibly. A model trained on data from one hospital, one scanner or one population may perform worse elsewhere. Errors have serious consequences, so choosing the right metric matters: recall is critical when missing a disease is dangerous. And doctors and patients need to understand and trust the system. That is why, in most real deployments, ML acts as decision support: it flags cases and prioritises work, but a qualified clinician makes the final decision.

Case study 3: recommender systems

Slide 48

Key points

- Predict missing entries in a user–item rating table
- Collaborative filtering: similar users like similar items
- Used by video, music and shopping platforms
- Mixes supervised and unsupervised ideas

Explanation

Every time YouTube suggests a video, an online shop shows “customers also bought”, or a music app builds a playlist for you, a recommender system is at work. Recommenders are among the most commercially important uses of ML.

The table shows the core idea in its simplest form. Rows are users, columns are items — here, films — and the numbers are ratings from 1 to 5. Most cells are empty, because each user has rated only a few items. The task is to predict the missing ratings, marked with question marks, and recommend the items with the highest predicted ratings.

One classic approach, collaborative filtering, says: users who agreed in the past will probably agree in the future. Look at Aziz and Dilnoza — they rated the first three films almost identically. Dilnoza loved the fifth film, so we predict Aziz will like it too. This is an interesting problem because it doesn't fit neatly into regression or classification; it is often described as a mix of supervised and unsupervised ideas.

Case study 4: computer vision — back to our “3”

Slide 49

Key points

- Image → 64 pixel features → classifier → digit
- 1,797 labelled digit images in scikit-learn
- Simple models (e.g. k-NN) typically reach >95% test accuracy
- Scaled-up versions: face unlock, postal codes, medical imaging, inspection

Explanation

Let's return to the problem we started with: recognising handwritten digits. This is a computer vision problem, and it is now easy to solve with machine learning.

The pipeline is: take the image, flatten its 8-by-8 grid into a list of 64 numbers — those are the features — then feed them into a classifier trained on labelled examples. The scikit-learn digits dataset contains 1,797 labelled images. Even a simple classifier such as k-nearest neighbours, which you will learn in Week 6, typically reaches well above 95 percent accuracy on unseen test images — without us writing a single rule about bumps or curves.

The same principle, scaled up with deep neural networks and millions of images, underlies face unlock on phones, reading of postal codes and bank cheques, medical image analysis and quality inspection on production lines. This is the power of learning from examples: the program we could not write by hand, we can now train in a few lines of code.

Project ideas: problems you could solve this semester

Slide 50

Key points

- Energy use prediction (regression)
- Air-quality prediction (regression)
- Crop yield or plant disease (regression / classification)
- Student at-risk prediction (classification; mind privacy)

- Diabetes / heart-disease risk (classification)
- Customer segmentation (clustering)
- Sources: Kaggle, UCI ML Repository, open-data portals

Explanation

Your midterm mini-project and final capstone project will ask you to apply ML to a real dataset. Start thinking now about problems that interest you. Here are some example directions — these are ideas, not assignments, and each would need a suitable public dataset.

Energy: predict household or building electricity consumption from weather and time — a regression problem. Environment: predict air-quality levels in a city from weather measurements. Agriculture: predict crop yield from rainfall, temperature and fertiliser use, or classify plant diseases. Education: predict whether a student is at risk of failing from attendance and assignment scores — classification — being careful with privacy. Health: predict diabetes or heart-disease risk using well-known public medical datasets. Business: segment customers of a shop into groups — clustering.

Good places to look for data are Kaggle, the UCI Machine Learning Repository and government open-data portals. In the lab today you will practise evaluating whether a dataset is suitable. In Week 2 there is a consultation specifically on choosing your midterm dataset.

Part 5 — The ML workflow

From a question to a working model — plus limits and ethics

In this part we look at machine learning as a process. Real ML projects are not just about choosing an algorithm: most of the time goes into defining the problem, collecting and cleaning data, and checking results honestly. We will walk through the seven steps of a typical workflow — the same steps you will follow in your midterm and capstone projects — and then discuss when not to use ML, its limitations, and responsible use.

The machine learning workflow in seven steps

Slide 52

Key points

- 1 Define the problem
- 2 Collect data
- 3 Prepare data
- 4 Train a model
- 5 Evaluate
- 6 Deploy
- 7 Monitor — and iterate back to earlier steps

Explanation

Here is the standard ML workflow. It is the backbone of this entire course and of your projects.

Step one: define the problem — what exactly are we predicting, and why? Step two: collect data. Step three: prepare the data — clean it, handle missing values, encode categories and split into training and test sets. Step four: choose and train a model. Step five: evaluate it on the test set with suitable metrics. Step six: deploy the model so it can be used. Step seven: monitor it over time, because the world changes.

Note the arrow going back. This is not a straight line; it is a cycle. Evaluation often shows that you need better features, more data or a different model, so you go back to earlier steps. Experienced practitioners expect to loop several times.

In this course, weeks 2 and 3 focus on steps two and three, weeks 4 to 11 on steps four and five, and week 12 on pipelines that make the whole process reproducible. Deployment and monitoring we will discuss conceptually.

Steps 1–2: define the problem and collect data

Slide 53

Key points

- Define: decision supported, exact target, type of task, success metric, baseline
- Collect: source & permission, label column, size, representativeness, privacy
- Problem understanding = 30% of the midterm rubric

Explanation

The first two steps are the ones beginners most often rush, and they cause most project failures.

To define the problem, answer a few concrete questions. What decision will this prediction support? What exactly is the target variable — for example, not “predict health” but “predict whether a patient will be readmitted within 30 days”? Is it regression, classification or clustering? How will you measure success — what metric, and what level would be useful? And would a simple rule or a basic average already be good enough? That last one gives you a baseline to beat.

For data collection, ask: where does the data come from and are you allowed to use it? Is there a label column? How many rows are there — for this course, a few hundred to a few tens of thousands of rows is ideal. Does the data represent the situations where the model will be used? And does it contain personal information that must be protected?

These questions form the “problem understanding” criterion, which is 30 percent of the midterm rubric.

Step 3: prepare the data — often the biggest job

Slide 54

Key points

- Garbage in, garbage out
- Missing values: drop or fill (e.g. median)
- Categorical features: encode as numbers (one-hot)
- Scaling: important for distance-based models (k-NN)
- Outliers & errors: detect and fix
- Train–test split before fitting anything, to avoid leakage

Explanation

Data preparation is often the most time-consuming step in real projects. Real data is messy, and models are only as good as the data we feed them — “garbage in, garbage out”.

Typical tasks are shown in this table. Missing values: some cells are empty, and we must decide whether to remove those rows or fill them in, for example with the median. Categorical features: models need numbers, so text categories like the district name must be encoded — for example with one-hot encoding. Scaling: features on very different scales, such as size in square metres and number of rooms, can confuse distance-based models like k-NN, so we rescale them. Outliers and errors: a house with a negative size or an age of 900 years is a data-entry error. Finally, the train-test split — and doing it before computing anything from the data, to avoid leakage.

You will practise the first two in Week 3, scaling in Week 6, and you will combine them all into a clean pipeline in Week 12.

Steps 4–5: train and evaluate

Slide 55

Key points

- Start simple and compare against a baseline
- Regression metrics: MAE, MSE, RMSE

- Classification metrics: accuracy, precision, recall, F1
- Confusion matrix: TN, FP, FN, TP
- Accuracy can mislead with unbalanced classes

Actual: not spam	48 True Neg	4 False Pos
Actual: spam	6 False Neg	42 True Pos
	Pred: not spam	Pred: spam

Figure: example confusion matrix for a spam filter (100 test e-mails).

Explanation

Once the data is ready, we choose a model and train it on the training set. For beginners, a good principle is: start simple. Train a very simple baseline first — for regression, predicting the average; for classification, predicting the most common class — and then check whether a real model beats it. If it doesn't, something is wrong.

Evaluation happens on the test set, and the right metric depends on the problem. For regression we use error measures like MAE — the average absolute error, easy to interpret in the units of the target — and RMSE, which punishes large errors more. For classification, accuracy is the simplest, but it can be misleading when classes are unbalanced. The confusion matrix on the right shows the four possible outcomes for a spam filter: true negatives, false positives, false negatives and true positives. From these we compute precision — of the e-mails flagged as spam, how many really were? — and recall — of all real spam, how much did we catch?

Don't worry if this feels fast. You will compute every one of these metrics by hand and in code in Weeks 4 and 5.

Steps 6–7: deploy and monitor

Slide 56

Key points

- Deploy: app, API, scheduled job or dashboard
- Data/concept drift: the world changes, performance decays
- Monitor performance and incoming data; retrain when needed
- In this course: a clean, reproducible notebook/pipeline (IWS 3, capstone)

Explanation

A model that stays in a notebook helps no one. Deployment means putting the model where it can be used: inside a web or mobile app, behind an API that other programs can call, in a scheduled job that scores new data every night, or even as a simple dashboard for decision makers.

After deployment, the work isn't finished. The world changes, and a model trained on last year's data may slowly get worse. This is called data drift or concept drift. Customer behaviour changes, new spam tactics appear, a hospital buys a new scanner, prices rise with inflation. So we monitor the model: track its performance and the distribution of incoming data, and retrain it when needed.

For this course, “deployment” means something simpler: a clean, reproducible notebook or pipeline that someone else can run from start to finish and get the same results. That is exactly what IWS 3 in Week 12 asks you to build, and it is also a criterion in the capstone rubric: “the notebook runs correctly and follows a clean, reproducible pipeline”.

When should — and shouldn't — you use ML?

Slide 57

Key points

- Good fit: complex patterns, changing rules, enough data, errors tolerable, repeated at scale
- Poor fit: simple rule works, little data, perfect explanation/accuracy required, dangerous errors without human oversight
- Midterm rubric: justify why your problem suits ML

Explanation

Machine learning is powerful, but it is not always the right tool. A good engineer knows when not to use it.

ML is a good fit when the pattern is too complex to write as rules, like recognising images or speech; when the rules change over time, like spam or fraud; when you have enough relevant, representative data; when some errors are acceptable, because every model makes mistakes; and when the task is repeated many times, so automation pays off.

ML is a poor fit when a simple rule or formula already works — you don't need ML to calculate tax or to convert Celsius to Fahrenheit; when you have little or no data; when every decision must be perfectly explained or perfectly correct, as with some legal requirements; and when mistakes would be dangerous and there is no human in the loop to catch them.

This judgement is part of the midterm rubric: you must explain why your problem suits machine learning.

Limitations and risks of machine learning

Slide 58

Key points

- Data quality: garbage in, garbage out
- Bias: models can learn and repeat historical unfairness
- Overfitting: memorising instead of generalising
- Explainability: some models are black boxes
- Correlation $\neq$ causation
- Limitations = 10% of the midterm rubric

Explanation

Every ML model has limitations, and identifying them is a required part of your projects — it is 10 percent of the midterm rubric, and the viva will ask you about at least one limitation.

Here are five of the most common. Data quality: wrong labels, missing values or unrepresentative samples produce wrong models. Bias: if historical data reflects unfair decisions, for example in hiring or lending, the model can learn and repeat that unfairness. Overfitting: a model that memorises training data fails on new data. Lack of explainability: some models, especially deep neural networks, are hard to interpret, which is a problem when decisions must be justified. And correlation is not causation: a model can find that two things occur together without knowing that one causes the other — ice-cream sales and drowning incidents both rise in summer, but ice cream doesn't cause drowning.

A good habit: for every model you build, write one or two sentences about what could go wrong and for whom.

Responsible and ethical machine learning

Slide 59

Key points

- Fairness: equal performance and fair errors across groups
- Transparency: explain how decisions are made
- Privacy: minimal, protected, anonymised data
- Accountability: humans stay responsible
- In projects: public/anonymised data, cite sources, academic integrity

Explanation

Because ML systems make or influence decisions about people, we have a responsibility to build them carefully. Four principles are widely accepted.

Fairness: check whether the model performs equally well for different groups — for example, men and women, or different regions — and whether its errors fall unfairly on one group. Transparency: be able to explain, at least in general terms, how the model reaches its decisions and what data it uses. Privacy: collect only the data you need, protect personal information, anonymise where possible and respect the law and your university's rules. Accountability: a human or organisation must remain responsible for the system's decisions; “the algorithm did it” is not an acceptable excuse.

For your own projects this has practical consequences. Use public or properly anonymised datasets. Don't include names, phone numbers or ID numbers. Mention possible bias in your limitations section. And follow academic integrity: cite datasets, libraries and any code you adapt. The university's policy on plagiarism applies fully to notebooks and code.

Part 6 — Tools & what's next

The Python toolkit, your first ML program, the lab and your projects

In the final part we move from ideas to tools. We will meet the Python libraries you will use all semester, write a complete machine learning program in about ten lines, learn the three-method pattern that every scikit-learn model follows, and then preview today's lab and the project expectations.

Your machine learning toolkit

Slide 61

Key points

- Python: the main language of ML
- Jupyter / Google Colab: notebooks mixing code, output and text
- NumPy: fast arrays and maths
- pandas: tables (DataFrames) for data work (Week 2)
- matplotlib: plots and charts
- scikit-learn: models, data splitting and metrics
- GitHub: version control and project submission

Explanation

These are the tools you will use every week. Python is the dominant language for machine learning because it is readable and has an enormous ecosystem of libraries.

Jupyter notebooks and Google Colab let you mix code, output, charts and explanatory text in one document — perfect for experiments and for your project reports. Colab runs in the browser with nothing to install, and it's free. NumPy provides fast arrays and mathematical operations; almost every other library is built on it. pandas gives you DataFrames — tables with named columns — for loading, cleaning and exploring data; it's the focus of Week 2. matplotlib draws plots and charts. scikit-learn provides ready-made implementations of all the ML models in this course, plus tools for splitting data and computing metrics. And GitHub, together with shared cloud folders, is where you will keep and submit your project code with version control.

In today's lab you will open Colab, check that all these libraries work, and run your first notebook.

Your first machine learning program

Slide 62

Key points

- Load data → split → choose model → fit → predict → evaluate
- Iris: 150 flowers, 4 features, 3 species
- Result: accuracy 1.0 on 30 test flowers — Iris is an easy dataset
- Be suspicious of perfect scores on real data

Explanation

Here is a complete machine learning program. It is only about ten lines, and by Week 5 you will understand every part of it in depth.

Line by line: we load the famous Iris dataset — 150 flowers, four measurements each, and three species as the label. We split it into 80 percent training and 20 percent test data; the `random_state` makes the split reproducible. We create a k-nearest-neighbours classifier that looks at the 5 closest training flowers. We call `fit` to train it. We call `predict` to classify the test flowers. And we compare the predictions to the true species with `accuracy_score`.

When I run it, the accuracy is 1.0 — all 30 test flowers correct. Before you get too excited: Iris is a small, clean and famously easy dataset. Real projects are much harder, and a perfect score on real data should make you suspicious of leakage or overfitting.

We will run this program live now, and you will run it yourself in today's lab.

The scikit-learn pattern: fit, predict, score

Slide 63

Key points

- 1. Create: `model = SomeModel(hyperparameters)`
- 2. Train: `model.fit(X_train, y_train)`
- 3. Predict: `model.predict(X_new)`
- 4. Score: `model.score(X_test, y_test)`
- Same pattern for every model → easy comparison (Week 10)

Explanation

One of the best things about scikit-learn is its consistent design. Every model in the library — linear regression, logistic regression, k-NN, decision trees, Naive Bayes, and even k-means — follows the same simple pattern.

First you create the model object, choosing its hyperparameters. Then you call `fit` with the training features and labels — this is the training loop we discussed. Then you call `predict` on new features to get predictions. And many models also have a `score` method that returns a default metric, such as accuracy for classifiers or R-squared for regressors.

This means once you learn the pattern, switching models is often a one-line change: replace `KNeighborsClassifier` with `DecisionTreeClassifier` and the rest of your code stays the same. That makes it very easy to compare models — which is exactly what you will do in Week 10 and in your capstone project. Unsupervised models like k-means follow the same pattern, except that `fit` takes only `X`, because there are no labels.

Today's lab (LC 1): tools, notebooks, datasets, projects

Slide 64

Key points

- A: Google Colab setup and saving to Drive
- B: notebook basics — cells, shortcuts, Restart & run all

- C: library check and Python warm-up
- D: load and explore Iris and a CSV dataset; first plot
- E: first ML model, dataset evaluation, project expectations
- Submit your notebook at the end

Explanation

Right after this lecture we have our first laboratory session. You will receive a lab handout, and there is a starter notebook to go with it.

The lab has five parts. In Part A you will open Google Colab, create your first notebook and learn to save it to Google Drive. In Part B you will learn how notebooks work: code and text cells, running cells, keyboard shortcuts, and the very important habit of “Restart and run all”. In Part C you will check that NumPy, pandas, matplotlib and scikit-learn are available and do a quick Python warm-up. In Part D you will load and explore real datasets — the built-in Iris dataset and a CSV file from the web — and make your first plot. And in Part E you will run the first ML program from this lecture, then learn how to evaluate whether a dataset is suitable for your mini-project, and go through the project expectations.

At the end you will submit your notebook. Please bring your laptop if possible, and make sure you can log in to your Google account.

Project expectations: what's coming

Slide 65

Key points

- W2 consultation: choose midterm dataset
- W3–W7: IWS 1 dataset preparation & EDA
- W7: midterm mini-project — notebook, report, 5–7 min demo (30%)
- W9–W15: capstone — dataset, comparison, report, presentation & viva (50%)
- Pairs/small groups; each member explains their part

Explanation

Let's look ahead at the projects, because they make up 80 percent of your grade and they start early.

In Week 2 there is a consultation on choosing your midterm dataset. From Week 3 you work independently on IWS 1: dataset preparation and exploratory analysis. The midterm mini-project is presented in Week 7: a notebook, a short report, and a 5 to 7 minute demonstration. It is graded on four criteria: problem understanding, model choice and evaluation at 30 percent each, and limitations at 10 percent.

After the midterm, you will choose a capstone dataset in Week 9, plan the project in Week 13, get support and rehearse in Week 14, and submit in Week 15, with the viva in the final exam week. You will work in pairs or small groups; the capstone rubric includes teamwork, and every member must be able to explain their own part of the work.

My advice: form your group early, choose a dataset you genuinely find interesting, and use the consultations.

Watch and read before next week

Slide 66

Key points

- Videos: StatQuest intro to ML; CGP Grey “How Machines Learn”; 3Blue1Brown neural networks (optional); AlphaGo – The Movie (optional)
- Géron (2022), Ch. 1
- James et al. (2023) ISLP, Ch. 1–2
- Müller & Guido (2016), Ch. 1
- Explore scikit-learn.org and Kaggle datasets

Explanation

Here are resources to consolidate today's lecture. First, four excellent free videos — the links are clickable in the slide. StatQuest's “A Gentle Introduction to Machine Learning” explains the core ideas with very simple examples. CGP Grey's “How Machines Learn” is an entertaining animated explanation of how learning systems are built and tested. 3Blue1Brown's “But what is a neural network?” is optional but beautiful if you're curious about deep learning. And “AlphaGo – The Movie” is a full-length documentary about the Go match, if you want an inspiring story for the weekend.

For reading, start with Chapter 1 of Géron's Hands-On Machine Learning, “The Machine Learning Landscape”, which covers everything we discussed today in more depth. Chapters 1 and 2 of An Introduction to Statistical Learning with Applications in Python give the statistical view. And Chapter 1 of Müller and Guido's Introduction to Machine Learning with Python is very practical and uses scikit-learn — ideal preparation for the labs.

Also, please explore scikit-learn.org and Kaggle's datasets page, and start noting datasets that interest you.

Key takeaways

Slide 67

Key points

- ML = learning rules from data + answers
- Learning = adjusting parameters to reduce error
- Generalisation to unseen data is the real goal; beware overfitting
- Supervised (regression, classification), unsupervised, reinforcement
- ML is a workflow; data quality matters most
- Know the limitations and act responsibly

Explanation

Let's summarise the six most important ideas from today.

One: machine learning means learning rules from data and answers, instead of writing rules by hand. Two: a model is a function with parameters, and learning means adjusting those parameters to reduce error on training data. Three: what really matters is generalisation — performance on new, unseen data — so we always evaluate on a separate test set and watch out for overfitting. Four: there are three main types of ML — supervised, which divides into regression and classification; unsupervised, such as clustering; and reinforcement learning. Five: real ML is a workflow, from defining the problem through data preparation, training, evaluation, deployment and monitoring — and data quality usually matters more than the choice of algorithm. Six: ML has real limitations and ethical responsibilities, and identifying limitations is part of every project you'll do.

If you remember only one sentence from today, make it this: machine learning is learning from examples to make good predictions on new data.

Exit ticket (3 minutes)

Slide 68

Key points

- Q1: Traditional programming vs ML in 1–2 sentences
- Q2: A problem from your life — regression, classification or clustering? What is the label?
- Q3: Why evaluate on data not seen during training?

Explanation

Before we finish, please take three minutes to answer these three questions on a piece of paper or in the course Teams channel. They are not graded, but they help me see what was clear and what needs revisiting next week.

Question one: in one or two sentences, explain the difference between traditional programming and machine learning. Question two: give an example of a problem from your own life or interests, and say whether it would be regression, classification or clustering — and what the label would be. Question three: why must we evaluate a model on data it has not seen during training?

Good answers will mention: rules written by hand versus rules learned from data and answers; a clear target variable of the right type; and the idea that we care about generalisation, and that testing on training data would reward memorisation or overfitting.

Closing remarks

Thank you for your attention. Let's take a few minutes for questions — about the content, the course organisation or the projects.

After the break we go to the computer lab for Lab 1. Remember to have your Google account ready. Next week, in Lecture 2, we will start working with data in Python using pandas: loading tables, viewing rows and columns and computing simple summaries — the foundation for everything else in this course. Please watch at least the StatQuest video and read Chapter 1 of Géron before then.

Appendix A — Glossary

Term	Definition
Machine learning (ML)	Building programs whose behaviour is learned from data rather than written as explicit rules.
Sample / instance	One example in a dataset — one row of the table.
Feature (X)	An input variable describing a sample — one column. Numerical or categorical.
Label / target (y)	The output we want to predict.
Model	The learned function that maps features to predictions, $\hat{y} = f(x; \theta)$.
Parameters (θ, w, b)	Numbers inside the model that are learned from data.
Hyperparameters	Settings chosen by us before training, e.g. k in k -NN or the maximum depth of a tree.
Training / fitting	Adjusting the parameters to reduce error on the training data (<code>model.fit</code>).
Prediction / inference	Using a trained model on new inputs (<code>model.predict</code>).
Loss / error	A number measuring how wrong the predictions are, e.g. mean squared error.
Gradient descent	An iterative method that repeatedly nudges the parameters in the direction that reduces the loss.
Training set / test set	Data used to learn the parameters / data held back to estimate performance on unseen data.
Generalisation	How well a model performs on new, unseen data — the real goal of ML.
Underfitting	The model is too simple and misses the pattern (high bias).
Overfitting	The model memorises the training data, including noise, and fails on new data (high variance).
Data leakage	Information from the test set sneaking into training, making results look better than they are.
Supervised learning	Learning from labelled examples; includes regression and classification.
Regression	Supervised learning where the target is a number.
Classification	Supervised learning where the target is a category (class).
Decision boundary	The border in feature space where a classifier switches from one class to another.
Unsupervised learning	Learning from unlabelled data to find structure: clustering, dimensionality reduction, anomaly detection.
Clustering / k-means	Grouping similar samples; k-means alternates assigning points to the nearest centre and moving centres to the mean.
Reinforcement learning	An agent learns a policy by acting in an environment and receiving rewards.

Term	Definition
Data / concept drift	Changes in the world that make a deployed model gradually less accurate.
Baseline	A very simple reference model (e.g. always predict the average) that a real model must beat.

Appendix B — Formula sheet (preview of Weeks 4–5)

Measure	Formula	Meaning
Linear model	$\hat{y} = w \cdot x + b$	Prediction as a straight line with slope w and intercept b
Mean absolute error (MAE)	$MAE = (1/n) \sum y_i - \hat{y}_i $	Average size of the error, in the units of y
Mean squared error (MSE)	$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$	Average squared error; punishes large errors
Root MSE (RMSE)	$RMSE = \sqrt{MSE}$	Like MSE but back in the units of y
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Fraction of correct predictions
Precision	$TP / (TP + FP)$	Of the items flagged positive, how many were right
Recall	$TP / (TP + FN)$	Of the real positives, how many were found
F1-score	$2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$	Balance of precision and recall

Worked example (confusion matrix from Slide 55: $TN = 48$, $FP = 4$, $FN = 6$, $TP = 42$): accuracy = $90/100 = 0.90$; precision = $42/46 \approx 0.913$; recall = $42/48 = 0.875$; $F1 \approx 0.894$.

Appendix C — Self-check questions

Q1. Explain the difference between traditional programming and machine learning.

Answer: Traditional programming: a human writes rules; data + rules → answers. ML: data + answers → a learning algorithm produces the rules (a model), which is then used to predict answers for new data.

Q2. State Mitchell's definition using T, E and P, and apply it to a spam filter.

Answer: A program learns from experience E with respect to task T and performance measure P if its performance on T , measured by P , improves with E . Spam: T = classify e-mails; E = labelled e-mails; P = % correctly classified on new e-mails.

Q3. How are AI, ML and deep learning related?

Answer: Nested: deep learning $\subset$ machine learning $\subset$ artificial intelligence. AI includes rule-based systems; ML learns from data; deep learning uses many-layered neural networks.

Q4. In a house-price table, what are the samples, the features and the label?

Answer: Samples = houses (rows); features = size, rooms, age, district (inputs X); label = price (y).

Q5. What is the difference between a parameter and a hyperparameter?

Answer: Parameters are learned from data (e.g. slope and intercept); hyperparameters are chosen before training (e.g. k in k -NN, tree depth).

Q6. Why do we evaluate on a separate test set?

Answer: To estimate performance on unseen data (generalisation). Evaluating on training data rewards memorisation and hides overfitting.

Q7. Describe underfitting and overfitting and how to recognise them.

Answer: Underfitting: too simple, high error on both training and test data. Overfitting: very low training error but much higher test error — the model memorised noise.

Q8. Is predicting tomorrow's temperature regression or classification? And predicting "rain / no rain"?

Answer: Temperature → regression (a number). Rain / no rain → classification (a category).

Q9. Give one example each of an unsupervised task and a reinforcement-learning task.

Answer: Unsupervised: grouping customers into segments (clustering) or detecting unusual transactions. Reinforcement: a robot learning to grasp objects from rewards; a program learning to play Go.

Q10. Name the seven steps of the ML workflow.

Answer: Define the problem → collect data → prepare data → train → evaluate → deploy → monitor (and iterate).

Q11. Give two situations in which ML is a poor choice.

Answer: When a simple rule already works (e.g. unit conversion); when there is little data; when every decision must be perfectly explained or correct; when errors are dangerous and no human checks them.

Q12. Name three limitations or risks of ML models.

Answer: Poor data quality; bias learned from historical data; overfitting; lack of explainability; confusing correlation with causation; drift after deployment.

Q13. What do fit, predict and score do in scikit-learn?

Answer: fit trains the model on training data; predict produces predictions for new inputs; score returns a default metric (accuracy for classifiers, R^2 for regressors).

Appendix D — Readings and videos

Textbooks (course literature)

- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media. — Chapter 1: The Machine Learning Landscape.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer. — Chapters 1–2.
- Müller, A. C., & Guido, S. (2016). *Introduction to Machine Learning with Python*. O'Reilly Media. — Chapter 1.

Videos

- StatQuest — A Gentle Introduction to Machine Learning:
https://www.youtube.com/watch?v=Gv9_4yMHfHl

- CGP Grey — How Machines Learn: <https://www.youtube.com/watch?v=R9OHn5ZF4Uo>
- 3Blue1Brown — But what is a neural network? (optional):
<https://www.youtube.com/watch?v=aircAruvnKk>
- DeepMind — AlphaGo: The Movie (optional, full documentary):
<https://www.youtube.com/watch?v=WXuK6gekU1Y>

Online resources

- Google Colaboratory — colab.research.google.com
- scikit-learn documentation — scikit-learn.org
- Kaggle datasets and notebooks — kaggle.com
- UCI Machine Learning Repository